DATABASE SCHEMA DOCUMENTATION

Web-Based ML Model Evaluator

This document describes the database schema implemented for the Web-Based ML Model Evaluator. The backend uses Flask-SQLAlchemy with SQLite as the default database, configurable through the DATABASE_URL environment variable.

1. Database Overview

The schema contains four tables: users, datasets, models, and evaluation_results. These tables represent user accounts, uploaded dataset metadata, machine-learning model records, and evaluation outputs.

- ORM: Flask-SQLAlchemy / SQLAlchemy
- Default database: SQLite
- Default database file: backend/ml_evaluator.db
- Initialization: init_db(app) registers the models and calls db.create_all().

Note: db.create_all() creates missing tables; it does not migrate existing tables when model definitions change.

2. users

Purpose: Stores user account information.

Column	Data Type	Constraints	Description
id	INTEGER	Primary key, NOT NULL	Unique user identifier.
username	VARCHAR(80)	UNIQUE, NOT NULL	Username.
email	VARCHAR(120)	UNIQUE, NOT NULL	Email address.
password_hash	VARCHAR(256)	NOT NULL	Hashed password value.
created_at	DATETIME	NOT NULL; server timestamp default	Record creation time.

3. datasets

Purpose: Stores dataset metadata and file location.

Column	Data Type	Constraints	Description
id	INTEGER	Primary key, NOT NULL	Unique dataset identifier.
user_id	INTEGER	FK to users.id, NOT NULL	Dataset owner.
filename	VARCHAR(255)	NOT NULL	Recorded filename.
file_path	VARCHAR(512)	NOT NULL	Path to the dataset file.
file_size	BIGINT	NULL allowed	File size, if recorded.
row_count	INTEGER	NULL allowed	Number of rows, if recorded.
column_count	INTEGER	NULL allowed	Number of columns, if recorded.
metadata_json	JSON	NULL allowed	Optional dataset metadata.
created_at	DATETIME	NOT NULL; server timestamp default	Record creation time.

Foreign keys: user_id → users.id

4. models

Purpose: Stores ML model and training configuration metadata.

Column	Data Type	Constraints	Description
id	INTEGER	Primary key, NOT NULL	Unique model-record identifier.
user_id	INTEGER	FK to users.id, NOT NULL	Associated user.
dataset_id	INTEGER	FK to datasets.id, NOT NULL	Dataset used by the model.
model_name	VARCHAR(128)	NOT NULL	Model name.

algorithm	VARCHAR(128)	NOT NULL	Selected algorithm.
model_path	VARCHAR(512)	NULL allowed	Optional saved-model path.
status	VARCHAR(32)	NOT NULL; default 'pending'	Model/training status.
config_json	JSON	NULL allowed	Optional model configuration.
created_at	DATETIME	NOT NULL; server timestamp default	Record creation time.

Foreign keys: user_id → users.id; dataset_id → datasets.id

5. evaluation_results

Purpose: Stores evaluation metrics and optional detailed outputs.

Column	Data Type	Constraints	Description
id	INTEGER	Primary key, NOT NULL	Unique evaluation identifier.
user_id	INTEGER	FK to users.id, NOT NULL	Associated user.
model_id	INTEGER	FK to models.id, NOT NULL	Evaluated model.
metrics_json	JSON	NOT NULL	Evaluation metrics.
confusion_matrix_json	JSON	NULL allowed	Optional confusion matrix.
report_json	JSON	NULL allowed	Optional detailed report.
created_at	DATETIME	NOT NULL; server timestamp default	Record creation time.

Foreign keys: user_id → users.id; model_id → models.id

6. Relationships

Parent table	Child table	Relationship
users	datasets	One user can own multiple datasets.
users	models	One user can have multiple model records.
datasets	models	One dataset can be associated with multiple model records.
users	evaluation_results	One user can be associated with multiple evaluation results.
models	evaluation_results	One model can have multiple evaluation results.

The schema includes user_id on models and evaluation_results. Application logic should ensure that users are authorized to access the datasets, models, and evaluations referenced by those records.

7. Entity Relationship Diagram (Textual)

```
USERS (1) ————————< DATASETS
USERS (1) ————————< MODELS >———————— (1) DATASETS
USERS (1) ————————< EVALUATION_RESULTS >———————— (1) MODELS
```

8. Implementation and Verification

The models are defined in backend/models/user.py, backend/models/dataset.py, backend/models/model.py, and backend/models/evaluation.py. Initialization is implemented in backend/database/db.py, which imports the models before calling db.create_all().

Verification performed: SQLAlchemy reported the four expected tables, and schema inspection confirmed their columns, nullability, data types, and foreign-key relationships. This confirms schema registration and inspection; it does not by itself establish that all CRUD operations or application routes pass.

9. Future Considerations

- Test create, read, update, and delete operations for each model.

- Test foreign-key behavior and validation of invalid references.
- Verify authorization so users cannot access another user's records.
- Use database migrations if the schema changes after deployment.
- Ensure password values are hashed and plaintext passwords are never stored.

10. Source Files

- backend/config.py
- backend/database/db.py
- backend/models/__init__.py
- backend/models/user.py
- backend/models/dataset.py
- backend/models/model.py
- backend/models/evaluation.py